

AI Agent Design Patterns

A Practical Guide for Developers

Single Agent · Sequential Agent · Parallel Agent

Open-source stack · Google Colab ready · Real-world examples

Introduction: Why Do We Need Agent Patterns?

Imagine you hired someone to help you plan a vacation. You could hire one very smart person and ask them to do everything — research flights, find hotels, look up restaurants, and check the weather. That works fine for a simple trip. But what if you were planning a large company retreat for 50 people across three cities? Suddenly, one person juggling everything becomes chaotic and unreliable.

The same logic applies to AI agents. When you give an AI a task to complete using tools like web search or APIs, how you structure that AI's "thinking" and "teamwork" makes a massive difference in reliability, speed, and maintainability.

This tutorial walks you through three fundamental patterns for designing AI agent systems, illustrated with working code from the companion Google Colab notebook.

What is an AI Agent?

An AI agent is not just a chatbot that answers questions. An agent is an AI that can:

- Receive a goal (not just a single question)
- Decide which tools to use to accomplish that goal
- Call those tools (web search, APIs, databases, calculators)
- Read the tool results and decide what to do next
- Loop through this process until the goal is achieved

Think of it like the difference between asking Google Maps 'how do I get to the airport?' versus hiring a driver. The driver does not just answer — they actively navigate, check traffic, find parking, and adjust in real time. An AI agent is that driver.

The Three Patterns at a Glance

Pattern	Real-world Analogy	Best For
Single Agent	One very smart employee doing everything	Simple, flexible tasks
Sequential Agent	An assembly line (each worker passes to the next)	Ordered, structured pipelines
Parallel Agent	A team of specialists all working simultaneously	Independent concurrent sub-tasks

Each pattern has trade-offs. By the end of this tutorial, you will know exactly which pattern to reach for depending on your problem.

The Tools We Use (All Open Source)

Tool	What It Does	Why We Use It
LangGraph	Orchestrates agents as a graph of nodes and edges	Open-source, flexible, production-grade
LangChain	Provides tool wrappers and LLM interfaces	Massive ecosystem of integrations
DuckDuckGo Search	Free web search, no API key required	Zero cost, great for learning
Groq + Llama 3	Fast, free-tier LLM running Meta's open Llama 3 model	Free tier, blazing fast inference



Before You Start

Get a free Groq API key at: <https://console.groq.com> (takes about 60 seconds).
Open the companion notebook `AI_Agent_Design_Patterns.ipynb` in Google Colab.
Run Step 1 (install) and Step 2 (API key setup) before any of the pattern sections.

Pattern 1: The Single Agent

The Real-World Analogy

Imagine a highly capable executive assistant. You walk up to them and say: "Plan a weekend trip to San Francisco for me." They figure out the rest — they search for flights, look up hotels, check restaurant reviews, and hand you a plan. You gave them one instruction. They reasoned through all the steps themselves.

That is exactly what the Single Agent pattern does. One AI agent, given a goal and a set of tools, reasons through what to do step-by-step entirely on its own.

How It Works: The ReAct Loop

The single agent uses a technique called ReAct (Reasoning + Acting). Every time the agent has a task, it enters this loop:

1. Reason — The LLM reads the task and decides: "What should I do first?"
2. Act — It calls a tool (e.g., run a web search)
3. Observe — It reads the tool's response
4. Repeat — Back to step 1 until it has enough to answer

The key point: the agent decides the sequence of steps itself. There is no hard-coded order. This makes it flexible — but also unpredictable for complex workflows.

The Code (Notebook: Pattern 1 cells)

In the notebook, the single agent is built with just two components:

Notebook: Pattern 1 — System Prompt

```
single_agent_system_prompt = """
You are a helpful travel planning assistant.
```

```
When a user asks you to plan a trip, use the search tool to:
```

- ```
1. Find top attractions at the destination
2. Find the best time to visit and current weather conditions
3. Find recommended restaurants
4. Find transportation options (flights, trains, etc.)
```

```
After gathering information, compile a clear trip plan with
sections for: Attractions, Dining, Travel Tips, and Getting There.
"""
```

The system prompt is the agent's "job description". It defines the role and the expected multi-step workflow. Notice that we are relying entirely on this text to enforce the order of operations — the agent reads it and decides how to proceed.

#### Notebook: Pattern 1 – Building and Running the Agent

```
from langgraph.prebuilt import create_react_agent

single_agent = create_react_agent(
 model=llm, # Llama 3 via Groq
 tools=[search_tool], # DuckDuckGo search
 state_modifier=single_agent_system_prompt
)

response = single_agent.invoke({
 "messages": [HumanMessage(content="Plan a trip to San Francisco.")]
})
```

The function `create_react_agent` does the heavy lifting. It builds a small `LangGraph` internally with two alternating nodes: an "agent" node (the LLM deciding what to do) and a "tools" node (executing whatever tool the LLM chose). These two nodes loop back and forth until the LLM stops calling tools.

#### What Happens When You Run This

- Step 1: LLM reads the task and system prompt. Decides to search for attractions.
- Step 2: DuckDuckGo search runs. Results come back.
- Step 3: LLM reads results. Decides to search for restaurants.
- Step 4: Another search. Another set of results.
- Step 5: LLM has enough information. Writes the final trip plan.

In the notebook output, you can trace every single step in the messages list.

## Strengths and Weaknesses

|             |  Strengths |  Weaknesses |
|-------------|-----------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| Simplicity  | Extremely easy to build — one function call                                                   | Entire logic lives in one prompt (gets unwieldy)                                               |
| Flexibility | Agent decides its own sequence of steps                                                       | Sequence is non-deterministic — can skip steps                                                 |
| Maintenance | One place to update the system prompt                                                         | Hard to debug when something goes wrong                                                        |
| Complexity  | Works great for simple multi-step tasks                                                       | Unreliable for tasks with many interdependencies                                               |

## When Should You Use a Single Agent?

### ✓ Use a Single Agent When

The task has 2–4 clear steps and they are mostly independent.

You can write the full logic in a system prompt under ~200 words.

The order of steps does not critically matter (e.g., searching for hotels vs. restaurants can happen in any order).

You are prototyping or building a proof of concept and speed of development matters most.

### ✗ Avoid a Single Agent When

The task has 5+ steps that must happen in a strict sequence.

Step B depends on the output of Step A — and Step A might fail or be skipped.

You need guaranteed, auditable execution order (e.g., for compliance or financial workflows).

You have found that the agent sometimes misses steps or does them in the wrong order.

## Pattern 2: The Sequential Agent

### The Real-World Analogy

Think of a car manufacturing assembly line. Car frame arrives at Station 1 → engine is installed. Frame moves to Station 2 → wheels are attached. Moves to Station 3 → paint is applied. The order is fixed. Station 3 never starts before Station 2 finishes. Every car that comes out is built the same reliable way.

The Sequential Agent pattern works exactly like this. You break a complex task into specialized sub-agents, each with one clear job. They run one after another. The output of Agent 1 becomes the input to Agent 2 — passed through a shared "scratch pad" that all agents can read and write.

### The Key Concept: Shared State (The Scratch Pad)

In the video, this is called the "shared session state". Think of it as a whiteboard in the middle of the room. Agent 1 walks in, writes their findings on the whiteboard, and leaves. Agent 2 walks in, reads what Agent 1 wrote, adds their own findings, and leaves. By the end, the whiteboard has everything.

In the code, this whiteboard is a Python dictionary called the state. Each agent receives the full state, reads what it needs, and returns updates to specific fields.

#### Notebook: Pattern 2 – Shared State Definition

```
from typing import TypedDict, Annotated
from langgraph.graph.message import add_messages

class TripPlanState(TypedDict):
 messages: Annotated[list, add_messages] # full chat history
 food_results: str # Agent 1 writes here
 transport_results: str # Agent 2 writes here (after reading food_results)
 user_query: str # passed through unchanged
```

Every field is accessible to every agent. The power comes from the fact that Agent 2 can explicitly read `food_results` before it searches for transportation — so it knows the restaurant area and can plan routes accordingly. This is the short-term memory of your agent system.

### The Two Specialized Agents

Instead of one general-purpose agent trying to do everything, we now have two laser-focused specialists:

## Agent 1: The Food Finder

### Notebook: Pattern 2 – Food Finder Agent

```
def food_finder_agent(state: TripPlanState) -> dict:
 query = state["user_query"]
 raw_search = search_tool.run(f"best restaurants {query}")

 # LLM synthesizes raw results into clean recommendations
 response = llm.invoke([HumanMessage(content=synthesis_prompt)])

 # Write findings to shared state – Agent 2 will read this
 return {
 "food_results": response.content,
 "messages": [AIMessage(content="Food Finder completed.")]
 }
```

Notice the return value. The agent only returns the fields it changes. LangGraph merges this partial update back into the full state automatically. This is clean, modular code — each agent only touches its own "lane".

## Agent 2: The Transportation Finder

### Notebook: Pattern 2 – Transportation Agent (reads from Agent 1)

```
def transportation_agent(state: TripPlanState) -> dict:
 query = state["user_query"]

 # KEY LINE: reads Agent 1's output from shared state
 food_context = state.get("food_results", "No food results available")

 # Uses food context to make smarter transport recommendations
 synthesis_prompt = f"""
 Context from food recommendations: {food_context[:500]}
 Now find transportation options for: {query}
 """
 response = llm.invoke([HumanMessage(content=synthesis_prompt)])

 return {"transport_results": response.content}
```

This is the "curly braces" concept from the video — Agent 2 dynamically injects Agent 1's output into its own prompt using Python's f-string. The result is a smarter agent that doesn't have to re-search from scratch.

## Building the Sequential Graph

This is where LangGraph shines. The graph structure makes the execution order a first-class concept — not buried in a system prompt, but explicit in the code:

**Notebook: Pattern 2 – Wiring the Sequential Graph**

```

from langgraph.graph import StateGraph, END

graph = StateGraph(TripPlanState)

Register each agent as a named node
graph.add_node("food_finder", food_finder_agent)
graph.add_node("transportation", transportation_agent)
graph.add_node("combine", combine_results)

EDGES enforce the strict sequential order – this is the key
graph.set_entry_point("food_finder") # always starts here
graph.add_edge("food_finder", "transportation") # always 2nd
graph.add_edge("transportation", "combine") # always 3rd
graph.add_edge("combine", END) # done

pipeline = graph.compile()

```

The edges in the graph are the assembly line conveyor belts. They physically enforce that `food_finder` always runs before `transportation`, no exceptions. There is no prompt-based wishful thinking here — it is structural.

**How to Inspect the Shared State**

After running the pipeline, the notebook shows you the full state contents — equivalent to the 'session state' tab in the video's ADK demo.

Run the 'Inspect the Shared State' cell in the notebook to see exactly what each agent wrote to `food_results` and `transport_results`.

**Strengths and Weaknesses**

|              | ✓ Strengths                                     | ⚠ Weaknesses                                   |
|--------------|-------------------------------------------------|------------------------------------------------|
| Control      | Execution order is guaranteed — no surprises    | Order is fixed — cannot adapt dynamically      |
| Reliability  | Each agent has one clear job — easy to test     | If Agent 1 fails, Agent 2 never runs           |
| Transparency | Shared state lets you audit what each agent did | State can grow complex with many agents        |
| Debugging    | Bugs are easy to isolate to a specific agent    | Adding a new step requires rewriting the graph |

## When Should You Use a Sequential Agent?

### ✓ Use Sequential Agents When

The task has a strict required order: Step B literally cannot start before Step A finishes.

Each step needs the output of the previous step as its input.

You need a reliable, repeatable, auditable pipeline (e.g., data processing, report generation).

Real example: Scrape product reviews → Sentiment analysis → Generate summary report.

Real example: Validate user input → Enrich with database lookup → Format for API submission.

### ✗ Avoid Sequential Agents When

The sub-tasks are independent and could run simultaneously (use Parallel instead).

The order might need to change based on the input (use Orchestrator pattern instead).

You only have 1–2 steps — a single agent is simpler.

## Pattern 3: The Parallel Agent

### The Real-World Analogy

You are planning a full day out in a new city. You need three things: museum recommendations, concert tickets, and restaurant reservations. If you do this yourself, you might Google each one sequentially — that takes time. But if you have three assistants, you can text all three at the same time. They each go and search independently. Fifteen minutes later, all three report back with answers, and you combine everything into your day plan.

That is the Parallel Agent pattern. Multiple specialized agents work simultaneously on independent sub-tasks. When they all finish, a final aggregator agent stitches the results together.

### The Fan-Out → Fan-In Structure

The parallel pattern has two phases, often described as fan-out and fan-in:

5. Fan-Out: A router dispatches work to all parallel agents at the same time
6. Fan-In: An aggregator waits for all parallel agents to finish, then synthesizes results

This structure is always combined with a sequential step at the end. The searching happens in parallel; the synthesis happens sequentially. This is the hybrid shown in the video.

### The Three Parallel Agents

In the notebook, we use a factory function to cleanly generate all three agents without repeating code. Each agent does exactly the same thing structurally — search and summarize — but for a different topic:

#### Notebook: Pattern 3 – Agent Factory (makes\_search\_agent)

```
def make_search_agent(topic: str, state_key: str, emoji: str):
 """Factory: returns a specialized agent function for `topic`."""
 def agent_fn(state: FullTripState) -> dict:
 query = state["user_query"]
 raw_results = search_tool.run(f"best {topic} in {query}")

 # Summarize with LLM
 response = llm.invoke([HumanMessage(content=summary_prompt)])

 # Write to this agent's dedicated state field
 return {state_key: response.content}
 return agent_fn
```

```
Instantiate 3 independent agents
museum_agent = make_search_agent("museums", "museum_results", "🏛️")
concert_agent = make_search_agent("concerts", "concert_results", "🎵")
restaurant_agent = make_search_agent("restaurants", "restaurant_results", "🍽️")
```

Each agent writes to its own dedicated field in the state — , , and . They never read from each other's fields because they have no dependencies on each other. This independence is what allows them to run in parallel safely.

## The Aggregator Agent

Once all three parallel agents finish, the aggregator reads all three results from the state and synthesizes them into a single cohesive plan:

### Notebook: Pattern 3 – Aggregator Agent

```
def aggregator_agent(state: FullTripState) -> dict:
 synthesis_prompt = f"""
 You are a master travel planner.
 Combine the following research into a day-by-day itinerary.

 MUSEUMS: {state['museum_results']}
 CONCERTS: {state['concert_results']}
 RESTAURANTS: {state['restaurant_results']}
 """
 response = llm.invoke([HumanMessage(content=synthesis_prompt)])
 return {"final_plan": response.content}
```

The aggregator only reads — it never searches. Its only job is to synthesize. This clean separation of concerns (search vs. synthesize) is a hallmark of well-designed agent systems.

## Building the Parallel Graph

The magic of LangGraph is in the edges. When you add multiple edges from the same source node, LangGraph executes the destinations concurrently:

### Notebook: Pattern 3 – Wiring the Parallel Graph

```
graph = StateGraph(FullTripState)

graph.add_node("router", router_node) # fan-out point
graph.add_node("museums", museum_agent)
graph.add_node("concerts", concert_agent)
graph.add_node("restaurants", restaurant_agent)
graph.add_node("aggregator", aggregator_agent) # fan-in point
```

```
graph.set_entry_point("router")

FAN-OUT: three edges from one node = concurrent execution
graph.add_edge("router", "museums")
graph.add_edge("router", "concerts")
graph.add_edge("router", "restaurants")

FAN-IN: aggregator waits for all three to complete
graph.add_edge("museums", "aggregator")
graph.add_edge("concerts", "aggregator")
graph.add_edge("restaurants", "aggregator")

graph.add_edge("aggregator", END)
pipeline = graph.compile()
```



### The Latency Math

Sequential: Total time = Time(museums) + Time(concerts) + Time(restaurants)

Example: Total time = 3s + 4s + 3s = 10 seconds

Parallel: Total time = MAX( Time(museums), Time(concerts), Time(restaurants) )

Example: Total time = MAX(3s, 4s, 3s) = 4 seconds

For N independent tasks, parallel execution is up to N times faster.

Run the timing comparison cell in the notebook to see this in action.

## Strengths and Weaknesses

|             | ✓ Strengths                                            | ⚠ Weaknesses                                     |
|-------------|--------------------------------------------------------|--------------------------------------------------|
| Speed       | Dramatically reduces latency for N independent tasks   | Higher token/API cost (N agents running at once) |
| Scalability | Adding a new sub-task = adding one node + two edges    | Requires a synthesis step, adding complexity     |
| Isolation   | Each agent's failure is isolated — others keep running | Not suitable if tasks depend on each other       |
| Clarity     | Each agent has one clear, auditable job                | State management becomes critical to get right   |

## When Should You Use a Parallel Agent?



### Use Parallel Agents When

The sub-tasks are truly independent — they don't need each other's output to start.

Reducing latency is important (users are waiting for real-time results).

Real example: A report generator that needs market data, competitor data, and internal sales data — all independent.

Real example: A content pipeline that translates the same document to French, Spanish, and German simultaneously.

Real example: A monitoring system that checks server health, database health, and API health all at once.

### **Avoid Parallel Agents When**

Task B needs Task A's output — use Sequential instead.

You have tight API rate limits (parallel calls may hit limits simultaneously).

The aggregation/synthesis step is complex enough to negate the speed gain.

## Bonus: The Hybrid Pattern (Parallel → Sequential)

### Combining Patterns for Real-World Workflows

In the real world, you rarely use just one pattern in isolation. The most powerful agent systems combine patterns strategically. The most common combination is Parallel fan-out followed by Sequential aggregation — and the companion notebook includes a reusable factory function that builds this hybrid dynamically.

Here is a concrete example of why you would combine them: imagine a competitive analysis tool. You want to research three competitors simultaneously (parallel, for speed). Then you want to generate a structured report: first write an executive summary, then write detailed findings, then write recommendations — each section building on the previous (sequential, for reliability). The result is a fast, structured, reliable workflow.

### The Hybrid Builder (Notebook: Bonus Cell)

```
 Notebook: Bonus – Dynamic Hybrid Pipeline Builder

def build_parallel_pipeline(search_tasks: list[tuple[str, str]]):
 """
 Accepts a list of (node_name, search_topic) pairs.
 Returns a compiled parallel-then-aggregate pipeline.
 """
 # All search agents run in parallel (fan-out)
 # The aggregator synthesizes results sequentially (fan-in)
 ...

Example: build a wellness trip planner in 5 lines
wellness_pipeline = build_parallel_pipeline([
 ("yoga", "yoga studios and wellness retreats"),
 ("spas", "luxury spas and day spas"),
 ("parks", "hiking trails and nature parks"),
])

result = wellness_pipeline.invoke({"user_query": "Sedona Arizona", ...})
```

The beauty of this builder is that you can swap in any set of search topics and get a parallel pipeline instantly. The graph structure, state management, and aggregation logic are all handled for you. This is the kind of reusable abstraction that makes agent-based systems maintainable at scale.

# Full Comparison and Decision Guide

## Pattern Decision Flowchart

Use this flowchart every time you start designing an agent system:

| Decision Flowchart                                                                                                                                                                 |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>Is the task simple and single-concern?</p> <ul style="list-style-type: none"> <li>└ YES → Use Single Agent</li> <li>└ NO → Continue...</li> </ul>                               |
| <p>Do the sub-tasks depend on each other's output?</p> <ul style="list-style-type: none"> <li>└ YES → Use Sequential Agent (A feeds into B)</li> <li>└ NO → Continue...</li> </ul> |
| <p>Can the sub-tasks run independently at the same time?</p> <ul style="list-style-type: none"> <li>└ YES → Use Parallel Agent + Aggregator</li> </ul>                             |
| <p>Does the order need to change based on the input?</p> <ul style="list-style-type: none"> <li>└ YES → Use Orchestrator pattern (Episode 2)</li> </ul>                            |
| <p>Does the output need quality review before delivery?</p> <ul style="list-style-type: none"> <li>└ YES → Use Loop + Critique pattern (Episode 2)</li> </ul>                      |

## Side-by-Side Summary

| Dimension        | Single Agent      | Sequential Agent   | Parallel Agent        |
|------------------|-------------------|--------------------|-----------------------|
| Build complexity | ★ Very Low        | ★★ Medium          | ★★★★ Higher           |
| Execution order  | Non-deterministic | Fixed / guaranteed | Concurrent            |
| Speed            | Medium            | Slowest (serial)   | Fastest               |
| Reliability      | Lower             | High               | High                  |
| Flexibility      | High              | Low (rigid)        | Medium                |
| API cost         | Lowest            | Medium             | Highest               |
| Best for         | Simple tasks      | Ordered pipelines  | Independent sub-tasks |
| Debuggability    | Hard              | Easy               | Medium                |

## Real-World Use Cases by Pattern

### Single Agent — Real Use Cases

- Customer support bot that searches a knowledge base and drafts a reply

- Personal assistant that checks your calendar, reads your emails, and gives a daily briefing
- Code reviewer that reads a file, searches documentation, and gives feedback

### **Sequential Agent — Real Use Cases**

- Content moderation pipeline: detect language → check toxicity → log results
- Invoice processing: extract data → validate against PO → approve or flag
- Onboarding flow: check eligibility → create account → send welcome email → log in CRM

### **Parallel Agent — Real Use Cases**

- Multi-language translation: translate the same document to 5 languages simultaneously
- Market research report: research market size, competition, and regulations concurrently
- System health dashboard: ping 10 different APIs/services in parallel every 30 seconds

## Quick Code Reference

Copy-paste these patterns as starting templates for your own projects.

### Pattern 1 — Single Agent Template

#### Template: Single Agent

```
from langgraph.prebuilt import create_react_agent
from langchain_groq import ChatGroq
from langchain_community.tools import DuckDuckGoSearchRun

llm = ChatGroq(model="llama3-8b-8192", temperature=0)
tools = [DuckDuckGoSearchRun()]

agent = create_react_agent(
 model=llm,
 tools=tools,
 state_modifier="You are a helpful assistant. Use tools to answer.",
)

result = agent.invoke({"messages": [HumanMessage(content="Your task here")]})
print(result["messages"][-1].content)
```

### Pattern 2 — Sequential Agent Template

#### Template: Sequential Agent

```
from typing import TypedDict, Annotated
from langgraph.graph import StateGraph, END
from langgraph.graph.message import add_messages

class MyState(TypedDict):
 messages: Annotated[list, add_messages]
 step1_result: str
 user_query: str

def agent_1(state: MyState) -> dict:
 result = do_step_1(state["user_query"])
 return {"step1_result": result}

def agent_2(state: MyState) -> dict:
 result = do_step_2(state["step1_result"]) # reads Agent 1's output
 return {"messages": [AIMessage(content=result)]}

graph = StateGraph(MyState)
graph.add_node("agent_1", agent_1)
graph.add_node("agent_2", agent_2)
graph.set_entry_point("agent_1")
```

```
graph.add_edge("agent_1", "agent_2")
graph.add_edge("agent_2", END)
pipeline = graph.compile()
```

## Pattern 3 — Parallel Agent Template

### Template: Parallel Agent

```
graph = StateGraph(MyState)
graph.add_node("router", router_fn) # fan-out
graph.add_node("task_a", agent_a)
graph.add_node("task_b", agent_b)
graph.add_node("task_c", agent_c)
graph.add_node("aggregator", aggregator_fn) # fan-in

graph.set_entry_point("router")

Fan-out: three parallel branches
graph.add_edge("router", "task_a")
graph.add_edge("router", "task_b")
graph.add_edge("router", "task_c")

Fan-in: all converge on aggregator
graph.add_edge("task_a", "aggregator")
graph.add_edge("task_b", "aggregator")
graph.add_edge("task_c", "aggregator")
graph.add_edge("aggregator", END)

pipeline = graph.compile()
```

## What's Next: Episode 2 Preview

The three patterns in this tutorial — Single, Sequential, and Parallel — give you the tools to build a huge variety of reliable agent workflows. But what happens when the workflow itself needs to be dynamic? When you don't know in advance which steps are needed? Or when an agent might produce a wrong answer and needs to self-correct? That's what Episode 2 covers.

| Episode 2 Pattern | What It Does                                                         | Real-world Analogy                                         |
|-------------------|----------------------------------------------------------------------|------------------------------------------------------------|
| Loop + Critique   | Agent generates output → Critic reviews it → Agent refines → Repeat  | An editor reviewing and rewriting a journalist's draft     |
| Orchestrator      | A coordinator agent dynamically routes tasks to the right specialist | A manager deciding which team member handles each task     |
| Agent as Tool     | One agent wraps another as a callable tool — infinite                | A surgeon (specialist) called in by a general practitioner |

|  |               |  |
|--|---------------|--|
|  | composability |  |
|--|---------------|--|

## Key Takeaways



### What You Learned in This Tutorial

Single Agent = Simple + flexible, but unreliable for complex ordered tasks.

Sequential Agent = Reliable + ordered, but rigid — great for assembly-line workflows.

Parallel Agent = Fast + scalable, but needs aggregation — great for independent sub-tasks.

Hybrid = Combine patterns to get the best of each (parallel search + sequential synthesis).

Shared State is the communication backbone — the scratch pad all agents read and write.

LangGraph makes execution order a first-class concept via graph edges, not just prompts.

Specialization is the key: each agent should do exactly one thing, and do it well.

## Further Reading

- LangGraph Documentation: <https://langchain-ai.github.io/langgraph/>
- LangGraph Multi-Agent Examples: [github.com/langchain-ai/langgraph/tree/main/examples/multi\\_agent](https://github.com/langchain-ai/langgraph/tree/main/examples/multi_agent)
- Groq Free Tier (Llama 3): <https://console.groq.com>
- ReAct Paper (the reasoning loop behind Single Agent): [arxiv.org/abs/2210.03629](https://arxiv.org/abs/2210.03629)
- Companion Notebook: `AI_Agent_Design_Patterns.ipynb` (provided with this tutorial)